

Asset Flip: a 2D Platformer

Project Technical Report

Patrick Anderson

andersonpm3@appstate.edu

Alan Ray

rayar@appstate.edu

Department of Computer Science
Appalachian State University

May 3, 2026

Abstract

Asset Flip is a two-dimensional side-scrolling platformer with melee combat, designed and implemented as a senior capstone project at Appalachian State University over the spring 2026 semester. The project was developed by a two-person team using the Godot game engine and the GDScript programming language. Its primary objective was to demonstrate competence with the discipline of game development by building a complete, playable artifact that exercises real-time physics simulation, input handling, finite state machines, signal-driven event architecture, scene management, audio mixing, and visual effects. A secondary objective was to manage scope and deliver iteratively across an eleven-sprint schedule.

The methodology was an agile, sprint-based workflow with clearly tiered features: foundational systems were prioritized so that the project would be playable from the earliest possible moment, with polish and additional content layered on as time allowed. The design centers on a small set of global singleton services that handle scene transitions, audio playback, screen-shake effects, health display, pause input, and score tracking, leaving each level scene responsible only for its own geometry and content. Every actor in the game, including the player character and all enemies, is driven by an explicit finite state machine; all enemies extend a common base class that consolidates shared logic such as hurt handling, knockback, edge detection, and death.

The final deliverable is a complete game containing a tutorial area, three sequential main levels, one secret level reachable through a hidden drop in the first level, eight enemy types, two boss encounters, a full audio bank with music and ambient layers, a pause menu, and complete win and death flows. All originally proposed essential features were delivered, along with most of the originally proposed stretch goals.

Keywords: game development, Godot Engine, GDScript, finite state machine, two-dimensional platformer, real-time physics simulation, agile development, signal architecture, scene graph, melee combat, sprint planning.

Table of Contents

Abstract.....	2
Table of Contents.....	3
List of Figures.....	4
List of Tables.....	4
1. Introduction and Background.....	5
1.1 Problem Statement.....	5
1.2 Background and Genre.....	5
1.3 Similar Existing Solutions.....	6
1.4 Solution Overview and Value.....	6
1.5 Problem Scope.....	7
1.6 Feature Summary and Use Cases.....	7
2. Design, Development, and Test.....	11
2.1 Design.....	11
2.1.1 System Architecture.....	11
2.1.2 Finite State Machines.....	11
2.1.3 Class Hierarchy.....	13
2.1.4 Combat and Signal Architecture.....	13
2.1.5 Scene Flow.....	13
2.2 Development.....	13
2.2.1 Sprint Plan and Order of Subsystem Development.....	13
2.2.2 Risk Management.....	13
2.2.3 Tooling.....	14
2.3 Testing.....	14
3. Results.....	15
3.1 Problems Encountered.....	17
3.2 Estimate Accuracy.....	17
4. Conclusions and Future Work.....	19
4.1 Summary.....	19
4.2 Lessons Learned.....	19
4.3 Utility of the Results.....	20
4.4 Suggested Future Work.....	20

List of Figures

Figure 1: Asset Flip main menu (title screen).

Figure 2: Level 1, the bright grassy area, with the player and a hedgehog enemy.

Figure 3: Level 2, the dark woods, showing demon and shroom enemies.

Figure 4: Level 3, the gothic castle, with the knight boss visible at center.

Figure 5: The Hollow Knight-inspired secret level reached through a hidden drop in Level 1.

Figure 6: Player finite state machine.

Figure 7: Generic enemy finite state machine (used by simple enemies).

Figure 8: Capstone milestone tracking radar chart.

List of Tables

Table 1: Sprint timeline showing planned and delivered work, sprints 1 through 11.

1. Introduction and Background

1.1 Problem Statement

The problem this project addresses is academic and technical: a two-person undergraduate team set out to acquire and demonstrate practical competence in real-time, interactive software development by shipping a complete video game. Game development is a useful vehicle for this learning because it forces the developer to confront several distinct categories of computer science problems in a single artifact. Building even a simple platformer requires reasoning about real-time physics simulation at a fixed timestep, input handling and event dispatch, animation state management, collision detection across heterogeneous shapes, finite state machines for character behavior, scene composition and resource management, audio mixing, and the ergonomics of a player-facing user interface. None of these are insurmountable individually, but integrating them into a coherent, playable product within a fixed timeline is the actual project, and the source of most of the engineering interest.

The intended users of the result are players of short-form action platformers and, secondarily, faculty and peers evaluating the project as a capstone deliverable. The context is a single-semester course at Appalachian State University with eleven one-week sprints. The principal constraints on solution choices were the team's starting experience level (neither author had shipped a game before), the time budget, and a deliberate decision to use only freely licensed third-party art and audio assets so that effort could be concentrated on engineering rather than content production.

1.2 Background and Genre

The two-dimensional platformer is one of the longest-running genres in video games. Its conventions were established in the mid-1980s by titles like Super Mario Bros. and refined for decades afterward. Modern entries such as Hollow Knight have layered tighter combat systems, more elaborate movement options, and more atmospheric art direction onto the same fundamental loop: a player-controlled character runs and jumps across two-dimensional terrain, defeating or avoiding enemies, in pursuit of a goal at the far end of each room or area.

Both Super Mario Bros. and Hollow Knight served as direct touchstones for this project. The bright, friendly first half of the game draws on the conventions of the former: cheerful palette, forgiving level design, simple enemies that hold a position or pace back and forth. The darker latter half and the secret area draw on the conventions of the latter: subdued palette, more deliberate

level design, ranged and aerial enemies, and a boss encounter with multiple attack patterns. The intentional contrast between these two halves gave the project both its working theme and its final name. The original working title was “CS Quest,” but during the early sprints one author found a free pack of bright, Mario-styled art assets while the other was working with a darker, Hollow Knight-styled pack. Rather than choose one, the team committed to using both, leaning into the tonal flip and renaming the project Asset Flip.

1.3 Similar Existing Solutions

As a genre, the two-dimensional platformer is well-served by commercial software. Mario titles, the Castlevania series, Hollow Knight, Celeste, Dead Cells, and hundreds of indie productions occupy this space. These existing solutions are highly polished, content-rich, and far beyond the scope of an eleven-week capstone. The strength of those products is years of iteration, professional art and audio teams, and broad distribution. Their weakness, from the standpoint of this project, is irrelevance as comparators: they are commercial products and not engineering exercises.

The relevant alternative for this project would have been to use a higher-level game framework or a visual programming environment such as GameMaker, Unity with a visual scripting plugin, or Construct. The choice of Godot 4 with hand-written GDScript was deliberate: it preserves a tight feedback loop without sacrificing access to the engine's underlying systems, and it leaves all interesting logic, including the finite state machines, signal wiring, and physics tuning, in the team's own code rather than hidden behind generated scaffolding.

1.4 Solution Overview and Value

The result is a self-contained game, distributed as a Godot project, runnable as a single executable on a desktop computer. A player launches the game, sees the title screen, presses Start, and is brought to a tutorial that introduces movement and combat. From there they progress through three sequential main levels with a tonal arc from bright and cheerful to dark and oppressive, fighting common enemies along the way. The third level ends in a fight against a knight boss with a layered move set. A hidden drop in the first level diverts to a fourth, secret level styled after Hollow Knight, ending in a fight against a witch boss with a different move set. Either boss, when defeated, leads to the win screen. Death at any point routes the player to a level-appropriate death screen with options to continue from that level, return to the main menu, or quit.

The value of the deliverable is twofold. As an artifact, it is a complete and playable demonstration of game engineering skill. As an exercise, it concretely exercises every category of system that any larger-scale interactive project would require: real-time simulation, input, state management, content pipelines, audio, and user interface plumbing.

1.5 Problem Scope

This project addresses the engineering of a single-player two-dimensional platformer with melee combat, four discrete level scenes, and two boss encounters. It does not address: networked multiplayer, save and load functionality, a settings or options menu, gamepad or controller input, accessibility features, internationalization, or three-dimensional rendering. Several of these were listed as stretch goals in the original proposal and were intentionally cut early in development to protect the core feature set. They are noted again in the Future Work section.

1.6 Feature Summary and Use Cases

The final feature set, condensed from the eleven sprint backlogs, is:

- Responsive two-dimensional movement with gravity, floor collision, walking, jumping, double jumping, dashing, and wall-collision reactions.
- Coyote time and jump-buffer windows of one hundred twenty milliseconds each, sized to make the controls feel forgiving without changing what is mechanically possible.
- Directional melee combat: side, upward, and downward attacks, with a downward attack that bounces the player off enemies, often called a pogo bounce.
- Nine enemy types: a worm, a hedgehog, a shroom, a generic dark cave enemy, a bat, a fly, a ranged demon, and two bosses (the knight and the witch).
- Two distinct boss fights, each with its own state machine and attack catalog.
- Four level scenes plus a tutorial, separated by transition zones.
- Visual effects: screen shake on impacts, a sword-slash effect, dash trail, and parallax-scrolling backgrounds in the dark areas.
- Audio: a three-track music bank, ambient loops for darker areas, footstep variation, and roughly twenty distinct sound effect categories drawing on multiple commercial-quality free packs.
- A complete user interface flow: main menu, pause menu, win screen, four death screens (one per level), and an on-screen health and score readout.

Two illustrative use cases follow. In the principal use case, a player launches the game, completes the tutorial, defeats a handful of common enemies in the grassy first level, progresses through the dark woods of the second level, reaches the cathedral that serves as the third level, defeats the knight boss, and is shown the win screen. In a secondary use case, a more exploratory player notices a suspicious gap in the grassy first level and drops down through it, bypassing the main path entirely and arriving in the Hollow Knight-styled secret area; they fight through the secret level and defeat the difficult witch boss to reach the same win screen by an alternate route.



Figure 1: The Asset Flip main menu. The painted background is a free asset; the title and buttons are rendered in-engine.



Figure 2: Level 1, the bright grassy area, showing the player (a jack-o-lantern character) and a hedgehog patrol enemy.

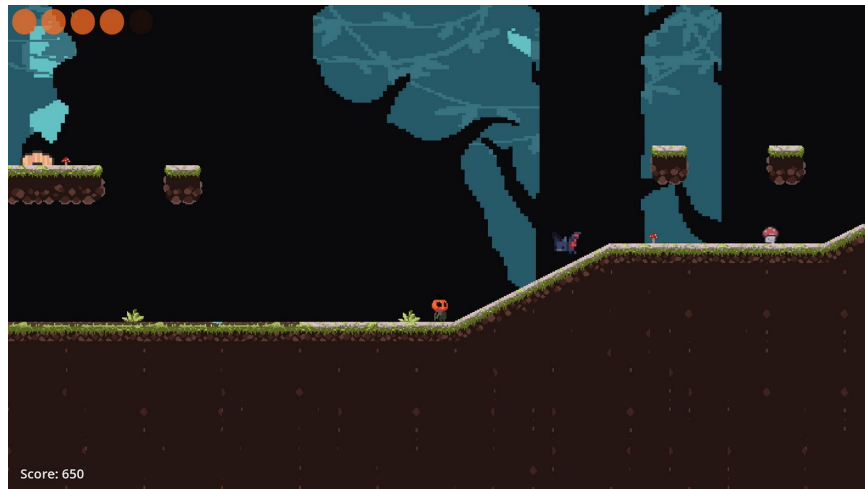


Figure 3: Level 2, the dark woods. Visible enemies include a bat (center), a worm (upper left), and a shroom (right).



Figure 4: Level 3, the gothic castle. The knight boss stands near the center of the screen at the start of combat.



Figure 5: *The Hollow Knight-inspired secret level, reachable through a hidden drop in Level 1. Note the light beam, dust motes, and the moodier palette.*

2. Design, Development, and Test

2.1 Design

2.1.1 System Architecture

Asset Flip uses two layers: persistent global services (Godot autoloads) and a single active scene tree. Autoloads persist across scene changes; the scene tree is rebuilt each transition. This separation defines the overall architecture.

Seven autoloads handle global concerns:

- GameFlow: scene transitions, fade overlay, win/death routing
- AudioManager: sound pools and music/ambient players
- VFX: screen shake via a trauma model, resolving the camera per scene
- HealthUI: CanvasLayer that reconnects to the current player
- PauseMenu: toggles pause and continues processing while paused
- ScoreManager: tracks score and emits updates
- UISound: auto-connects button hover/click sounds

The scene tree contains only level-specific content (geometry, enemies, player, transitions). Because global systems already exist, levels require minimal setup.

2.1.2 Finite State Machines

All actors use explicit finite state machines implemented uniformly: an enum, a state variable, and a per-frame match. Transitions go through a `set_state` method.

The player has six states: GROUNDED, AIR, ATTACK, HURT, DASH, DEAD, with movement, input locking, and transitions defined per state (e.g., DASH overrides gravity; DEAD routes through GameFlow).

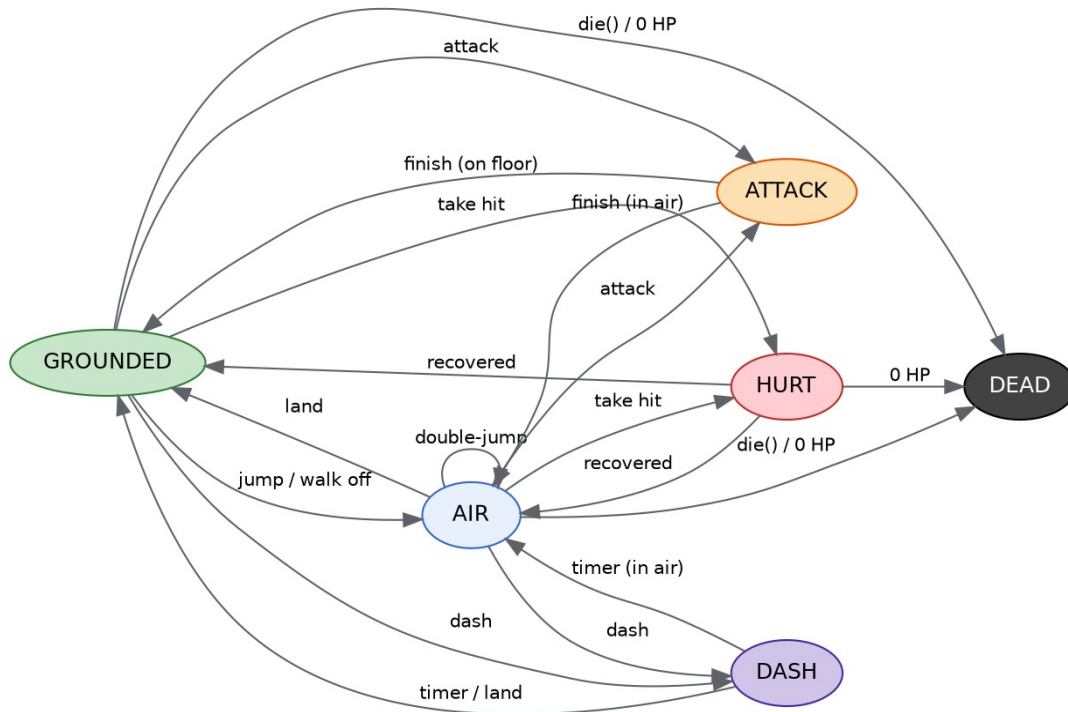


Figure 6: Player finite state machine. Take-hit and recovery edges connect both the grounded and aerial states to the central HURT state, and either of those can transition to DEAD on lethal damage.

Standard enemies use IDLE, CHASE, ATTACK, HURT, DEAD, while patrol-only types reduce this to PATROL, HURT, DEAD.

Bosses are more complex. With 11 states, the knight has multiple attacks, leap + plunge, an evasive roll, and randomized attack selection. With 10 states, the witch uses position-based tuning variables (far/medium/overhead/strong) for behavior control.

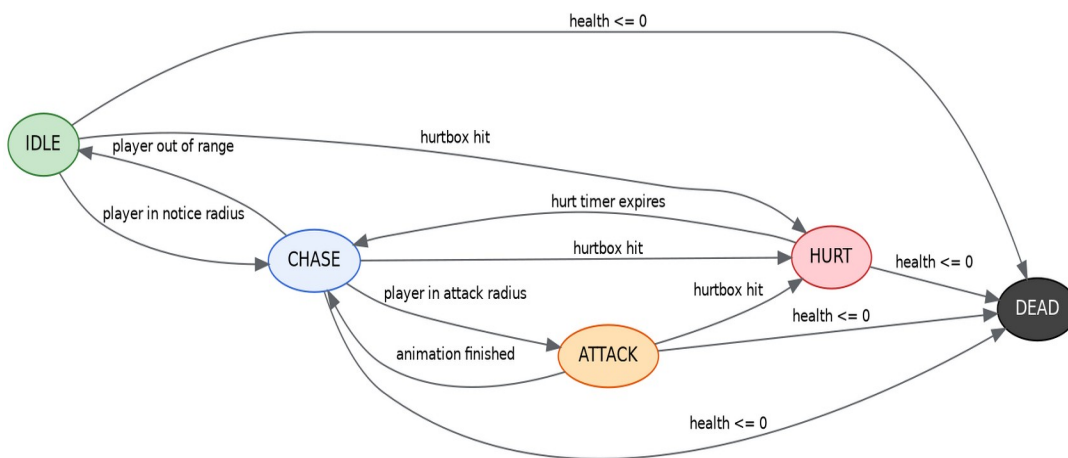


Figure 7: Generic enemy finite state machine. Patrol-only enemies use a reduced version with PATROL substituted for IDLE and CHASE.

2.1.3 Class Hierarchy

A shared base class (BaseEnemy) centralizes common behavior: hurt handling, knockback, edge detection, contact damage, and death logic. Subclasses override hooks such as `_enemy_ready` and `_enemy_physics_process`. This refactor reduced duplication and simplified enemy creation.

2.1.4 Combat and Signal Architecture

Combat uses signal-driven hit detection with separate hitbox and hurtbox Area2D nodes. When overlap occurs, the hurtbox emits a signal carrying the attacker's position. This design decouples attacker and receiver, allows frame-accurate hitbox activation, and supports multiple attack types without special-case logic. Actors handle damage, knockback, and state transitions locally.

2.1.5 Scene Flow

All transitions go through `GameFlow.go_to_scene`: fade out, change scene, reposition the player, update audio, fade in. Helper methods handle death and victory routing. A `last_level_path` variable ensures the continue button returns to the correct level.

2.2 Development

2.2.1 Sprint Plan and Order of Subsystem Development

The project ran over 11 one-week sprints. Core systems were built first to maintain a continuously playable build.

- **Sprints 1–4**: movement, combat, animation, state machines → playable prototype
- **Sprints 5–7**: first boss, more enemies, transitions, BaseEnemy refactor
- **Sprints 8–9**: win/lose states, UI, VFX, tutorial
- **Sprint 10**: final assets, layouts, audio
- **Sprint 11**: second boss, polish, tuning

All planned work was completed on schedule.

2.2.2 Risk Management

Three primary risks were identified at the start of the project: scope creep, underestimation of engine complexity, and the team's starting inexperience with GDScript and the Godot scene model. Each was addressed by a deliberate practice. Scope was managed by tiering features into essential, desirable, and stretch buckets, and by treating Sprint 4 as the deadline for a minimum

playable build; every later sprint was additive. Engine complexity was managed by allocating Sprint 1 almost entirely to tutorials and exploratory programming. Inexperience was addressed by pair-style review of any system that touched physics or signals, and by maintaining a running design document that captured non-obvious lessons as they were learned.

2.2.3 Tooling

The Godot Engine 4 served as both runtime and authoring environment. GDScript was the only language used in the project. Git, accessed primarily through GitHub Desktop, was used for version control; the repository was hosted on GitHub. Photopea was used for occasional asset creation in the case of light beams and dust motes.. Audacity was used to trim audio clips and convert formats, primarily into the OGG Vorbis format that Godot prefers.

2.3 Testing

Testing was manual and playtest-driven, as correctness is largely experiential.

Approach:

- repeated playtesting
- maintaining a checklist of recurring issues

Key examples:

- Movement: edge-case inputs (jump/dash interactions)
- Combat: multi-hit prevention, dead-enemy guards
- Enemies: full state coverage in isolation
- Bosses: animation timing, cooldown correctness
- Transitions: spawn placement, audio switching
- Audio: repeated triggers, pause behavior
- Difficulty: iterative tuning via full playthroughs

Common failure patterns led to standardized fixes:

- deferred property changes (`set_deferred`)
- ensuring death animations are non-looping
- avoiding duplicate `queue_free` calls

3. Results

This section evaluates the final deliverable against the original project proposal. The proposal listed essential, desirable, and stretch-goal features. All essential features were delivered. Almost all desirable features were delivered. Three stretch goals were intentionally cut early in development; the remainder were delivered. The single biggest deviation from the proposal is structural rather than feature-based: the proposal described two areas, and the final game has four. The one-area-of-light, one-area-of-darkness concept survived, but it was implemented as a tutorial plus three sequential levels with a tonal arc and a hidden fourth secret level, rather than two larger areas.

Table 1: Final feature delivery against the original project proposal.

Feature (proposed)	Tier	Status / notes
Responsive movement and jumping	Essential	Delivered. Coyote time and jump-buffer windows of one hundred twenty milliseconds.
Gravity and floor collision	Essential	Delivered.
Double jump and dash	Essential	Delivered. Dash overrides gravity for two hundred milliseconds.
Directional melee attack system	Essential	Delivered. Side, up, and down attacks; downward attack pogo bounces from enemies.
Player finite state machine	Essential	Delivered. Six states.
Enemy finite state machine	Essential	Delivered. Five states for chasing enemies, three for patrol enemies, ten and eleven for the bosses.
Enemy health and death	Essential	Delivered, consolidated into BaseEnemy.begin_death.
Room-to-room transitions	Essential	Delivered via TransitionArea nodes and GameFlow.go_to_scene.
Player health UI	Essential	Delivered as a HealthUI autoload that

		reconnects to the player on each scene load.
Two distinct areas (tonal flip)	Essential	Delivered, expanded into Tutorial plus Level 1, 2, 3, and a secret Final Level.
Menu screen	Essential	Delivered, with Start and Exit options.
Visual effects (screen shake, hit flash)	Essential	Delivered. VFX autoload with trauma-based screen shake.
Audio (music, SFX, ambient)	Essential	Delivered. AudioManager autoload with music, ambient, and a pool of eight SFX players.
Win and lose conditions	Essential	Delivered. Win screen on boss defeat; one death screen per level with continue-from-here support.
Boss fight with unique behavior	Desirable	Delivered, twice. Knight (eleven states) and witch (ten states).
Parallax scrolling backgrounds	Desirable	Delivered using Godot 4's Parallax2D nodes.
Three to twelve enemy types	Desirable	Seven enemy types delivered, plus two bosses.
Tutorial prompts	Desirable	Delivered as tutorial level.
Screen shake and knockback	Desirable	Delivered.
Boss health bar	Desirable	Cut as a stretch.
Controller support	Stretch	Cut. Keyboard only.
Ranged attacks (player)	Stretch	Cut. Player has melee only; the demon enemy is ranged.
Settings menu	Stretch	Cut. The Options button on the main menu is intentionally inert.
Save and load	Stretch	Cut.

--	--	--

3.1 Problems Encountered

Several recurring categories of problem accounted for most of the time spent on bug fixing during the project. The most frequent was a class of Godot-specific runtime errors caused by mutating physics-related properties (an Area2D's monitoring or monitorable flags, or a CollisionShape2D's disabled flag) directly inside a physics callback or signal handler. The fix is uniform: defer the change with `set_deferred` or `call_deferred`. After the third or fourth instance, this was promoted to a standing rule in the project's internal documentation.

A second recurring class of bug came from the interaction between an enemy's state machine and its death animation. The original implementation had each subclass handle its own death by playing an animation and calling `queue_free` in a per-subclass `_on_anim_finished`. When `BaseEnemy.begin_death` was added to consolidate the logic, the central method also connected `animation_finished` to `queue_free`; subclasses that still had a death case in their own animation handler then called `queue_free` twice. The fix was to remove all per-subclass death handling and rely entirely on `begin_death`, but the bug recurred a few times before that pattern was fully established.

A third class of bug, less frequent but more confusing while it lasted, involved the autoloads. Because autoloads initialize before any scene loads, code in their `_ready` that depends on a scene node, such as the camera in `VFX.gd`, will return null. The fix was a lazy-lookup pattern that caches the camera the first time it is needed, and an explicit warning in the project's internal documentation to never assume a scene node exists at autoload `_ready` time.

A fourth, more nuanced category showed up only after the death screens were introduced. The continue button on each death screen used `GameFlow.current_level_path` to return the player to the level they died in. This worked the first time, but on the second death the path returned was the death screen's own path, because `go_to_scene` had already overwritten `current_level_path` on the way to the death screen. The fix was to introduce a separate `last_level_path` variable that snapshots the previous gameplay scene before the death-screen transition, and to have the continue buttons read from that instead.

3.2 Estimate Accuracy

The sprint plan in the original proposal was substantially accurate. All eleven sprints were delivered on schedule. The plan's risk-mitigation principle of building a playable prototype by

Sprint 4 held up; the project was fully playable from that point onward, and every later sprint added rather than repaired. The most significant deviation was the addition of the Hollow Knight-styled secret level, which was not in the original plan but emerged organically from the asset-pack situation that gave the project its name. The second boss (the knight) was added in Sprint 11 as a polish pass and was completed within the sprint.

Three originally stretch-goal features were cut on purpose and never attempted: controller support, a settings menu, and save and load functionality. Each was judged early to be less interesting per unit of effort than additional levels, enemies, or polish, and that judgment proved correct in retrospect.

Team No (Members):		
Project initiation and proposal	20	out of 20
Planning and design	20	out of 20
Development and implementation	20	out of 20
Testing and evaluation	20	out of 20
Documentation and presentation	20	out of 20
	Raw	%
Total score from 100	100	100.00%

Figure 8: Capstone milestone tracking radar chart, showing progress across the five evaluation axes.

4. Conclusions and Future Work

4.1 Summary

Asset Flip set out to demonstrate practical competence with the discipline of game development by shipping a complete, playable two-dimensional platformer. Over eleven weeks, a two-person team using the Godot Engine 4 and the GDScript language built that game from a blank project to a polished deliverable containing a tutorial, three sequential main levels, one secret level, eight enemy types, two boss encounters, a full audio bank, a complete user interface flow, and an internally consistent visual identity. All originally proposed essential features were delivered. Most originally proposed desirable features were delivered. Three stretch goals were intentionally cut to protect the core.

4.2 Lessons Learned

Several lessons from this project generalized well beyond it. The first is that ordering subsystem development by dependency, rather than by interest, produces the smoothest projects. A playable prototype by Sprint 4 meant that every subsequent sprint operated on a working game; this dramatically reduced the cost of any individual change and made the project resilient to the inevitable surprises that real-time interactive software produces.

The second is that signal-driven, decoupled architectures are worth their initial complexity. Hit detection in Asset Flip routes through hitboxes and hurtboxes that emit named signals, and the recipient script handles those signals; the attacker never knows what it hit, and the recipient never knows where the hit came from. This made it straightforward to introduce new attackers without modifying any other code.

The third is the value of consolidating shared logic into a base class once the duplication becomes obvious. The BaseEnemy refactor in Sprint 7 collected hurt-timer ticking, knockback, edge detection, contact damage, and death into a single location. Every later enemy was easier to write; every later bug fix only had to be applied once.

The fourth is that real-time interactive software has a class of subtle, callback-related correctness bugs (deferred-mutation rules, double-free patterns, autoload initialization order) that are not obvious from any one of the engine's individual error messages. Maintaining a written checklist of these as they were discovered shifted the team from re-encountering them to defending against them, and was probably the single highest-leverage workflow change made during the project.

4.3 Utility of the Results

As an artifact, Asset Flip is a complete game. As an exercise, it demonstrates competence across every category of system that an interactive software project of any larger scale would require: simulation, input, state management, content pipelines, audio, and user interface. As a base for future work, the codebase is structured to absorb additional content (more enemies, more levels, more attacks) with minimal change to the surrounding architecture. The team's internal design and lessons-learned documentation, which grew alongside the code, is itself a useful deliverable for any team member or future student picking up similar work.

4.4 Suggested Future Work

- Settings and options menu, including audio volume sliders, key remapping, and a fullscreen toggle. The infrastructure for this exists in Godot but was not exposed in this build.
- Controller support. Godot's input map abstracts away the device, but additional design work is needed to handle simultaneous input from keyboard and controller cleanly, and to surface visual prompts that match the active device.
- A save-and-load system using checkpoints at level boundaries, with player health, score, and current level persisted to disk.
- A ranged attack option for the player, with separate animation, sound, and projectile logic.
- A parry mechanic that, on a tight input window before an enemy attack lands, cancels the attack and opens a counterattack window.
- More elaborate enemy artificial intelligence, including basic platforming so non-flying enemies can navigate vertical terrain rather than only patrolling flat ground.
- A boss health bar, deferred from the original desirable list. The infrastructure is largely the same as the player health user interface but with a different node layout.
- Interactive items in the environment: chests, breakable scenery, and pickups for currency or temporary buffs.
- Movement polish that was on the original stretch list: variable jump height tied to button hold time, terminal fall speed, and a configurable hang time at the apex of the jump arc.

References

- [1] Game Programming Patterns: State. [Online]. Available:
<https://gameprogrammingpatterns.com/state.html>
- [2] Godot Engine, Godot Engine documentation, version 4. [Online]. Available:
<https://docs.godotengine.org/>
- [3] Godot Engine, GDScript reference. [Online]. Available:
<https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/>
- [4] Helton Yan, Pixel Combat (audio asset pack). Available via itch.io. [Online].
- [5] Kenney, Impact Sounds (audio asset pack). [Online]. Available:
<https://kenney.nl/assets/impact-sounds>
- [6] Kenney, Interface Sounds (audio asset pack). [Online]. Available:
<https://kenney.nl/assets/interface-sounds>
- [7] P. Kostka and University of Games, Finite state machines, Medium. [Online]. Available:
<https://medium.com/university-of-games/finite-state-machines-87a6a88c1399>
- [8] Nintendo, Super Mario Bros. (video game). Released for the Nintendo Entertainment System, 1985.
- [9] Cyangmou, SunnyLand (visual and audio asset pack). Available via itch.io. [Online].
- [10] DavidKBD, Purgatory Pack (music). Available via itch.io. [Online].
- [11] Team Cherry, Hollow Knight (video game). Released for personal computer and console platforms, 2017.